

Лабораторная 3. Простые сортировки и QuickSort

Что должна гарантировать сортировка

После сортировки элементы расположены по неубыванию, а их количество и состав не изменились. Проверить только порядок недостаточно: функция, возвращающая пустой список, тоже не содержит нарушений порядка. Поэтому тестируются два свойства: упорядоченность и совпадение мультимножеств до и после. Лабораторная требует пузырьёк, вставки, выбор и рандомизированный QuickSort, а также счётчики операций и исследование стабильности.[1]

Сравнение ключей — проверка вида $<$, $>$ или $<=$. Обмен меняет местами две ячейки. Перемещение в нашем счётчике — одна запись в ячейку массива: обмен даёт два перемещения, присваивание локальной переменной не считается. Поэтому числа `swaps` и `moves` нельзя использовать как синонимы. Сравнения границ циклов также не входят в `comparisons`.

Пузырёк: большое значение идёт вправо

Соседние элементы сравниваются слева направо. Если левый больше правого, они меняются местами. После полного прохода максимальный элемент текущего участка оказывается в его конце. Этот конец можно исключить из дальнейшей работы.

Пример $[3, 1, 2]$: сравниваем 3 и 1 $\rightarrow [1, 3, 2]$; сравниваем 3 и 2 $\rightarrow [1, 2, 3]$. Последняя позиция уже окончательна. Если следующий проход не сделал обменов, весь оставшийся участок упорядочен и работу можно закончить. Этот флаг даёт $\Theta(n)$ в лучшем случае, но в среднем и худшем остаётся $\Theta(n^2)$.

Равные ключи не меняются: условие обмена строгое $>$. Поэтому пара равных элементов никогда не пересечётся, и сортировка стабильна. Замена $>$ на $>=$ разрушила бы это свойство и заставила бы напрасно менять равные элементы.

Вставки: как сортируют карты в руке

Слева поддерживается уже отсортированный префикс. Очередной элемент запоминается в `value`. Пока слева стоят большие элементы, они сдвигаются вправо; в образовавшееся место вставляется `value`.

Для $[3, 1, 2]$ вставка 1 превращает начало в $[1, 3]$, затем вставка 2 сдвигает 3 и даёт $[1, 2, 3]$. Инвариант: перед итерацией i участок $a[0:i]$ отсортирован и содержит те же элементы, что в начале обработки этого участка.

На упорядоченном массиве каждая вставка почти сразу останавливается: $\Theta(n)$. На обратном массиве приходится сдвигать $1+2+\dots+(n-1)$ элементов: $\Theta(n^2)$. Число сдвигов равно числу инверсий — пар, стоящих в неправильном порядке. Поэтому вставки полезны на коротких и почти отсортированных данных.

Выбор: минимум оставшейся части

На шаге i ищем минимум от i до конца и меняем его с $a[i]$. Для поиска минимума всегда нужно просмотреть весь остаток, даже если он уже отсортирован. Число сравнений точно $n(n-1)/2$, поэтому лучший, средний и худший случаи — $\Theta(n^2)$. Обменов при этом не больше $n-1$.

Выбор нестабилен. В $[(2,a), (2,b), (1,c)]$ первый обмен приводит к $[(1,c), (2,b), (2,a)]$. Равные ключи 2 поменяли взаимный порядок, хотя непосредственно друг с другом не сравнивались для обмена. Стабильность — свойство всего преобразования.

QuickSort и разбиение на три части

Выбираем случайный опорный ключ `pivot`. Все элементы группируются в три области: меньше `pivot`, равные ему, больше `pivot`. Середина уже на правильном месте относительно остальных; сортировать рекурсивно нужно только левую и правую части. Трёхчастное разбиение особенно полезно при множестве повторов.[2]

В реализации четыре участка: $[lo, left)$ содержит меньшие ключи; $[left, current)$ — равные; $[current, right]$ ещё не исследован; $(right, hi]$ — большие. Если текущий ключ меньше, отправляем его в левую часть и двигаем обе левые границы. Если больше — меняем с правым краем и уменьшаем `right`; `current` пока не двигаем, потому что прибывший элемент ещё не проверен.

Опорная позиция выбирается через `rng.randrange`. Seed 45 задаётся один раз для всего опыта. На каждом вызове генератор продолжает своё состояние: разные повторы могут иметь разные разбиения, но при повторном запуске весь поток выборов воспроизводится.

При достаточно сбалансированных разбиениях уровней порядка $\log n$, а суммарная работа уровня $O(n)$: ожидаемое время $O(n \log n)$. При последовательном выборе крайних по величине опорных элементов сумма работ $n + (n-1) + \dots$ даёт $O(n^2)$. Рандомизация уменьшает вероятность систематически плохих разбиений, но не превращает ожидаемую оценку в гарантию худшего случая.

В коде рекурсивно обрабатывается только меньшая часть, а большая — продолжением цикла. Каждый новый кадр получает не больше половины текущих элементов, поэтому стек ограничен $O(\log n)$, даже если суммарное время оказалось квадратичным. На массиве из одинаковых ключей трёхчастное разбиение заканчивает работу за $\Theta(n)$.

Сводка свойств

Алгоритм	Лучший случай	Средний	Худший	Доп. память	Стабильность
Пузырёк с флагом	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	Да
Вставки	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	Да
Выбор	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	Нет
QuickSort, 3 части	$\Theta(n)$ на равных	$O(n \log n)$, ожидаемо	$O(n^2)$	$O(\log n)$	Нет

Для QuickSort на различных ключах лучший порядок — $\Theta(n \log n)$. Его линейный лучший случай в таблице относится к равным ключам, которые исключаются сразу. Это особенность конкретной реализации.

Эксперимент и работа с файлами

```
python -m pytest labs/lab03 -q
python -m labs.lab03.experiment --variant 15
```

Из файлов варианта 15 берутся массивы размеров 100, 250, 630, 1600, 4000; упорядоченная и обратная версии строятся из того же набора значений. Это пять примерно геометрических точек, достаточных для сравнения роста без чрезмерного времени квадратичных сортировок. Перед каждым запуском берётся свежая копия вне таймера, иначе второй повтор измерял бы уже отсортированный результат.

Счётчики записываются при отдельном запуске. У QuickSort их случайная траектория может отличаться от пяти временных повторов; это отмеченная контрольная траектория, а не усреднённое число сравнений. Время включает ведение счётчиков у всех четырёх реализаций.

Вопросы на защите и ответы

1. Почему у выбора $\Theta(n^2)$ на любом входе? Он всегда ищет минимум полного остатка. Сумма длин остатков фиксирована и равна $n(n-1)/2$; порядок значений её не меняет.

2. Что такое стабильность и где она нужна? Равные по ключу записи сохраняют исходный взаимный порядок. Это полезно при сортировке записей сначала по одному полю, затем по другому. Пузырёк стабилен из-за строгого условия обмена; вставки не сдвигают равный ключ.

3. Откуда $O(n \log n)$ у QuickSort? У сбалансированного дерева разбиений логарифмическая глубина, а на каждом уровне суммарно обрабатывается $O(n)$ элементов. Для случайного pivot это ожидаемая оценка.

4. Как возникает худший случай и помогает ли случайность? Если pivot постоянно оказывается минимумом или максимумом, одна часть почти не уменьшается. При детерминированном первом pivot упорядоченный вход легко создаёт такой сценарий. Случайный выбор делает его маловероятным, но теоретически возможным.

5. Почему после обмена с right индекс current не увеличивается? Прибывший с правого края элемент ещё неизвестен. Если сразу перейти дальше, он может остаться в неправильной группе.

6. Почему одинаковые элементы не ломают реализацию? Все они попадают в среднюю часть и больше не сортируются. В тесте 12 000 одинаковых элементов выполняются 24 000 сравнений ключей и не возникает глубокая рекурсия.

7. Можно ли сравнивать только число обменов? Нет: выбор делает мало обменов, но много сравнений; вставки используют сдвиги вместо парных обменов. Нужно учитывать модель стоимости и время.

8. Что защищать по графикам? Показать линейный лучший случай пузырька и вставок, квадратичные кривые на трудных входах, более медленный рост QuickSort. Не объявлять случайно стабильный результат доказательством стабильности алгоритма.

Источники

[1] МТУСИ. КИМ-03, 2026.

[2] Robert Sedgewick, Kevin Wayne. Quicksort, материалы авторов Algorithms, 4th edition.

Отчёт по ЛР 3. Простые сортировки и QuickSort

Цель и выполненная работа

Реализованы алгоритмы и проверки, описанные в конспекте. Эксперимент выполнен на данных варианта 15, seed 45. В таблицах приведены фактические результаты запуска; время указано в секундах, если не оговорено иное.

Условия и методика

CPU: 12th Gen Intel(R) Core(TM) i7-12650H. ОС: Windows-11-10.0.26200-SP0. Python 3.12.12, NumPy 2.5.3, Matplotlib 3.11.1. Дата и время UTC: 2026-09-10T18:26:52.030797+00:00.

Один прогрев и пять повторов на точку, итог — медиана `time.perf_counter`. Ввод, генерация и подготовка свежего состояния исключены из таймера. Фоновые процессы и питание не контролировались; задержки рабочего компьютера могут влиять на отдельные точки. Контрольные суммы данных находятся в `environment.json`.

Результаты и их объяснение

Вход	Алгоритм	Наклон
random	bubble_sort	2.0355
random	insertion_sort	2.019
random	selection_sort	2.03858
random	quick_sort	1.18866
sorted	bubble_sort	1.06748
sorted	insertion_sort	1.05984
sorted	selection_sort	2.0436
sorted	quick_sort	1.20928
reversed	bubble_sort	2.02525
reversed	insertion_sort	2.02831
reversed	selection_sort	2.0315
reversed	quick_sort	1.17387

На случайных входах наклоны трёх простых сортировок около 2; у QuickSort — 1,189, что согласуется с ростом $n \log n$ на исследованном интервале. На упорядоченных входах пузырьрёк и вставки имеют наклоны 1,067 и 1,060, а выбор сохраняет квадратичное число сравнений.

Вход	Алгоритм	Сравнения	Обмены	Записи	c
random	bubble_sort	7994840	4005670	8011340	2.78059
random	insertion_sort	4009663	0	4009669	1.28176
random	selection_sort	7998000	3994	7988	1.50099

Вход	Алгоритм	Сравнения	Обмены	Записи	c
random	quick_sort	85671	38755	77510	0.0286277
sorted	bubble_sort	3999	0	0	0.0008319
sorted	insertion_sort	3999	0	3999	0.0013169
sorted	selection_sort	7998000	0	0	1.5281
sorted	quick_sort	91334	33994	67988	0.0275994
reversed	bubble_sort	7998000	7997996	15995992	3.92268
reversed	insertion_sort	7998000	0	8001995	2.5481
reversed	selection_sort	7998000	2000	4000	1.52486
reversed	quick_sort	81635	28872	57744	0.0269057

Счётчики — отдельный контрольный запуск; временные повторы QuickSort используют следующие состояния того же генератора случайных чисел. Время всех четырёх алгоритмов включает ведение счётчиков.

Алгоритм	Результат для [(2,a),(2,b),(1,c)]	Сохранён порядок
bubble_sort	[[1, 'c'], [2, 'a'], [2, 'b']]	True
insertion_sort	[[1, 'c'], [2, 'a'], [2, 'b']]	True
selection_sort	[[1, 'c'], [2, 'b'], [2, 'a']]	False
quick_sort	[[1, 'c'], [2, 'b'], [2, 'a']]	False

Пузырёк и вставки сохраняют a перед b; выбор и QuickSort переставляют эти метки. Пример доказывает нестабильность последних; стабильность первых обоснована строгими условиями сдвига и обмена в конспекте.

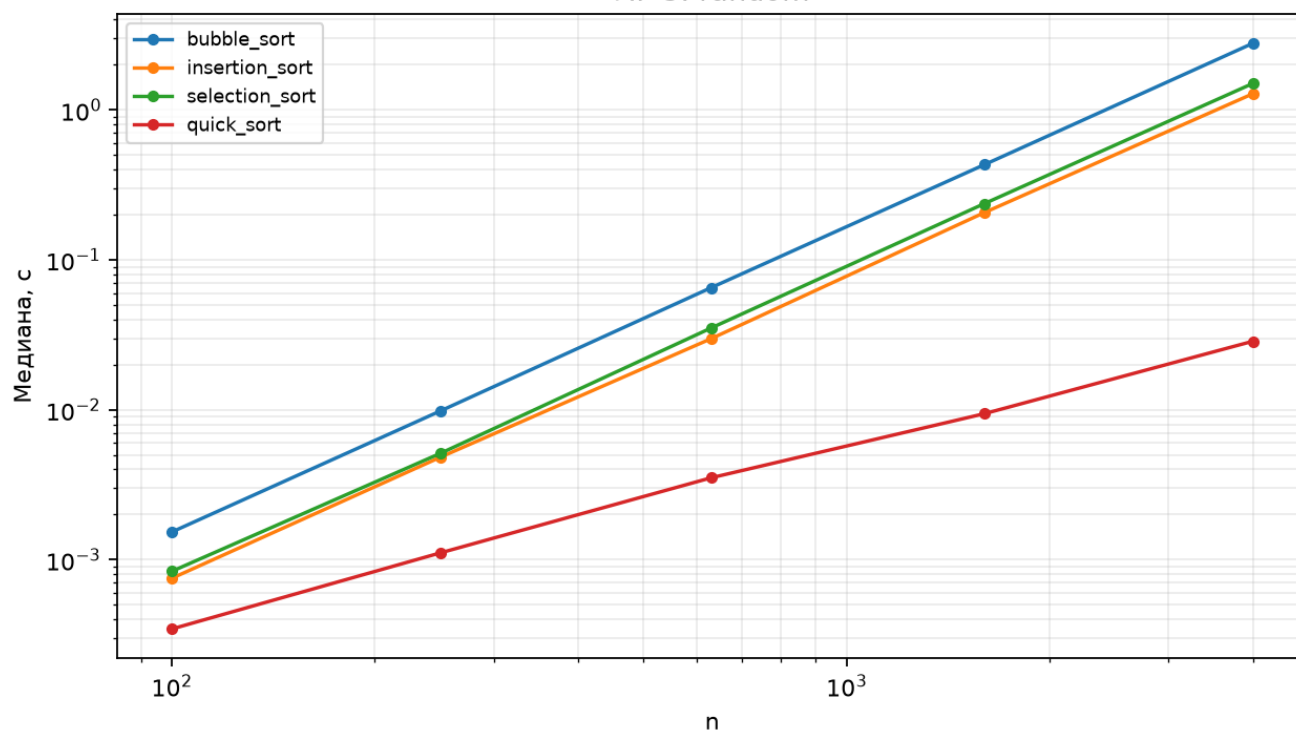
Проверка и вывод о применимости

Проверки находятся в test_lab.py. Запуск: `python -m pytest labs/lab03 -q`. Применены граничные случаи, инварианты и независимые эталоны; состав тестов подробно разобран в конспекте. Проверки всего комплекта также сохранены в verification.txt в корне репозитория.

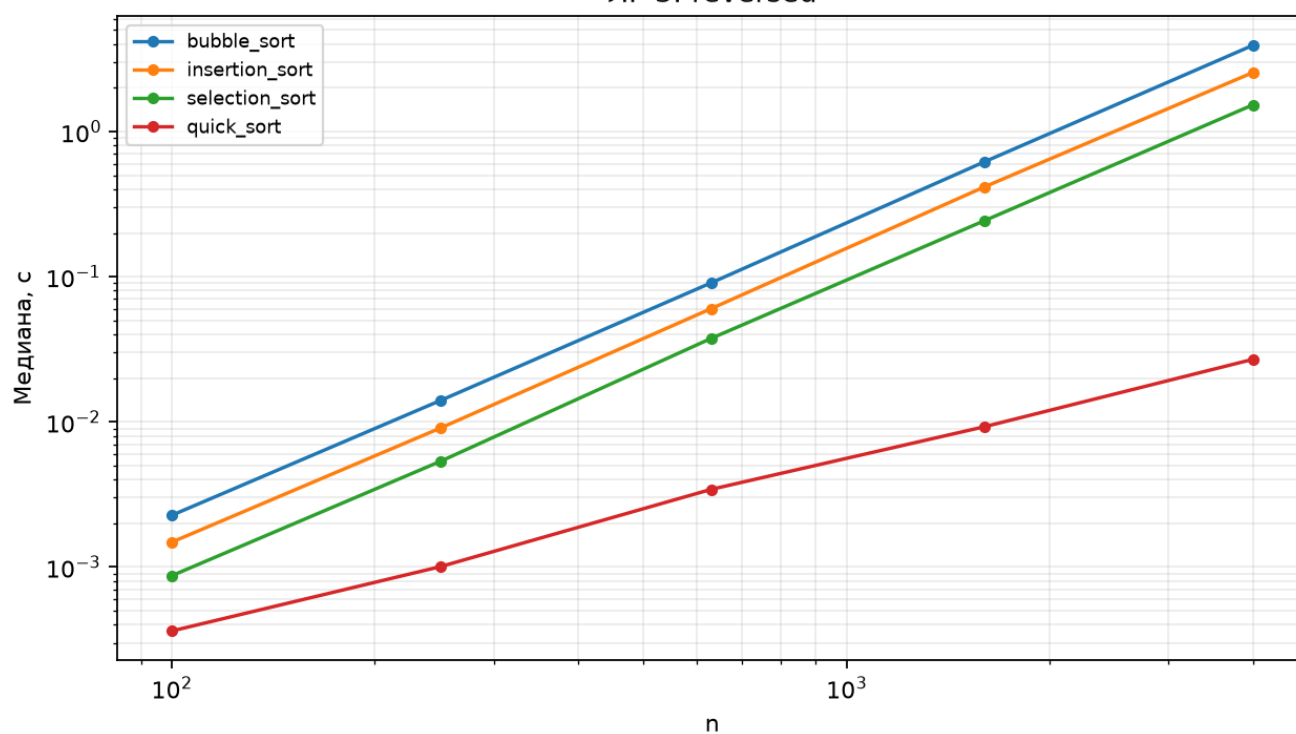
Вывод: принять для учебного использования в рамках явно описанных контрактов. Измерения подтверждают основные тенденции роста; ограничения реализаций и сравнения указаны выше и в конспекте. Автоматическая проверка не оценивает готовность к устной защите.

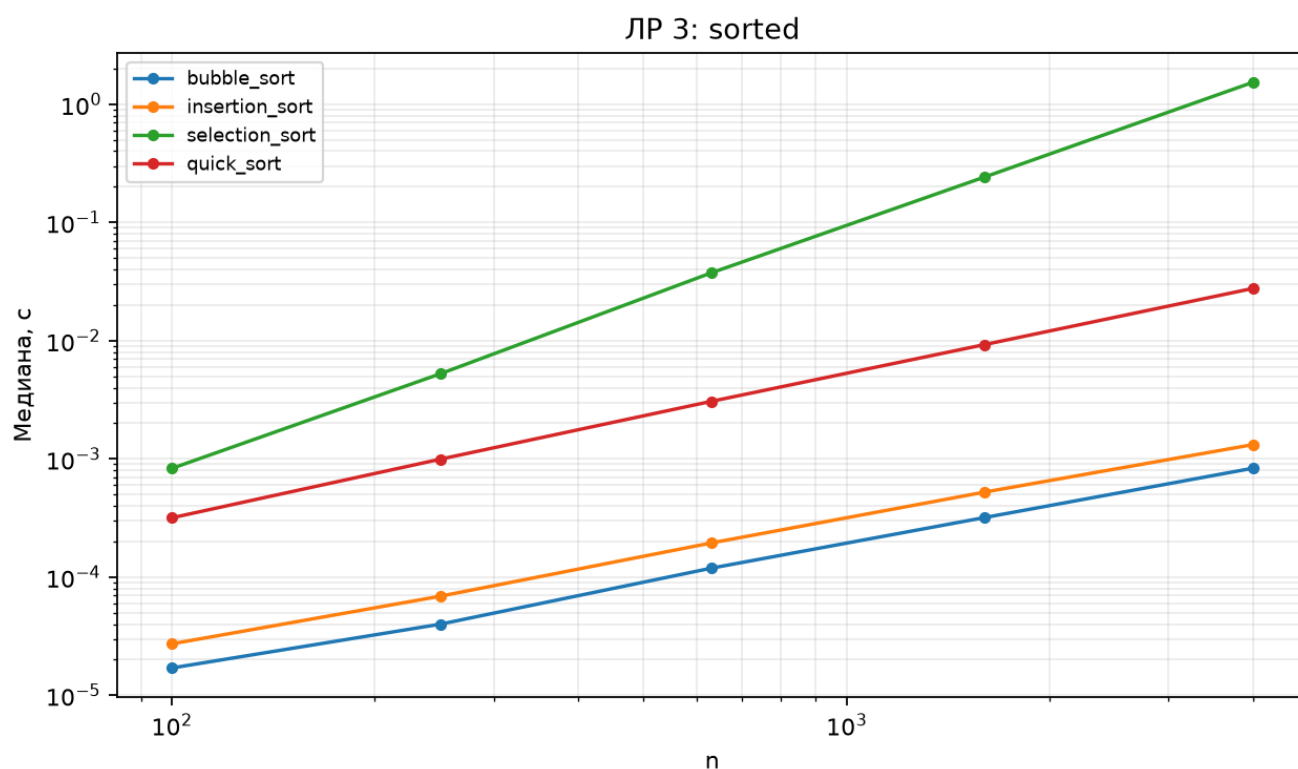
Графики

ЛР 3: random



ЛР 3: reversed





Исходные данные отчёта

CSV содержит медиану и все пять длительностей trial_1...trial_5. Вспомогательные таблицы без времени содержат точные счётчики или свойства структуры.

- measurements.csv
- slopes.csv
- stability.csv
- Паспорт окружения и контрольные суммы